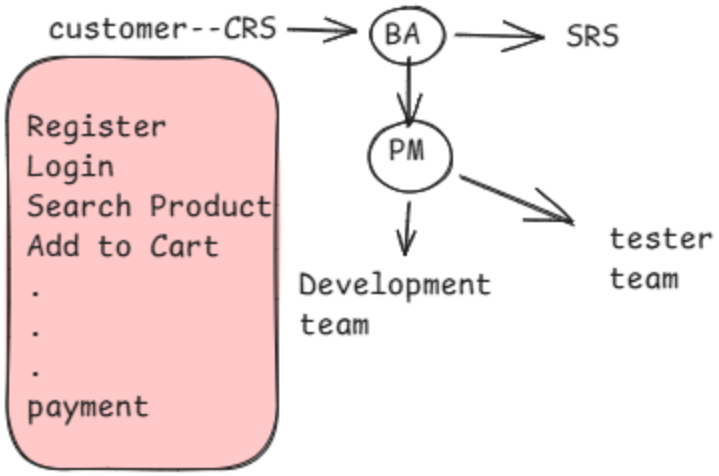
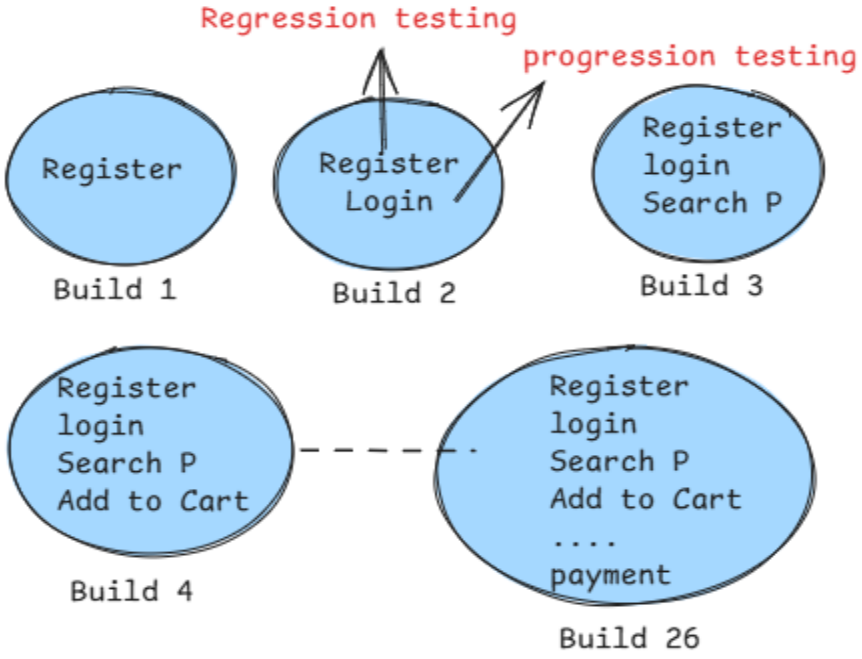
playwright

drawback of manaul testing

1)it takes more time
2)it requires more manpower
3)it is tedious and monotonous job
4)it is less accurate

Scenario : shopping application





Automation testing

Testing the application with the help of tool
or
converting the manual test cases into automation script

Why Automation is required

-->Regression testing has to be done for every build

When do we start automation?

1)After the application/build is stable
2)after the manual testcase is written and executed atleast once
3)when requirement are not changing frequently
4)when the testcase are repetitive
5)for regression testing

Advantage

1)less man power
2)less time
3)more accurate
4)faster execution
5)better quality

Disadvantage

1)initial development effort
2)tool knowledge required
3)programming language is required
4)maintenance

What is not Automable?

Exploratory Testing
Ad-hoc Testing
Usability Testing
CAPTCHA
Frequently Changing Features
Fingerprint/Biometric Authentication
Face Recognition
Human Judgment

Different tool

1)selenium-->free and open source
-->mainly used for web application
-->support chrome,edge,safari,firefox
-->support 14+ such as java ,python ..etc
-->support windows,macos,linux

2)cypress-->free and open source
-->used for for web application
-->support chrome,edge,firefox
-->support JS/TS
-->support windows,macos,linux

3)qtp -->(Quick test professional)
-->UFT(unified functional testing)
-->licensed (paid)
-->customer support
-->support VBScript
-->support web and desktop application
-->support chrome,edge,firefox

4)Appium -->free and open source
-->mainly used for mobile application testing
-->support Andriod and ios
-->supports java,python,JS,C#

5)test complete-->licensed (paid)
-->customer support
--> support web and desktop and mobile application
-->support windows

Playwright

playwright is an free and open-source tool by Microsoft
for automating web browser and testing web application
(released in 2020)

or

playwright is framework and also playwright is library for automating web browser, enabling end-to-end testing

or

Beyond browser automation,playwright offers a dedicated api for testing and automating web api

or

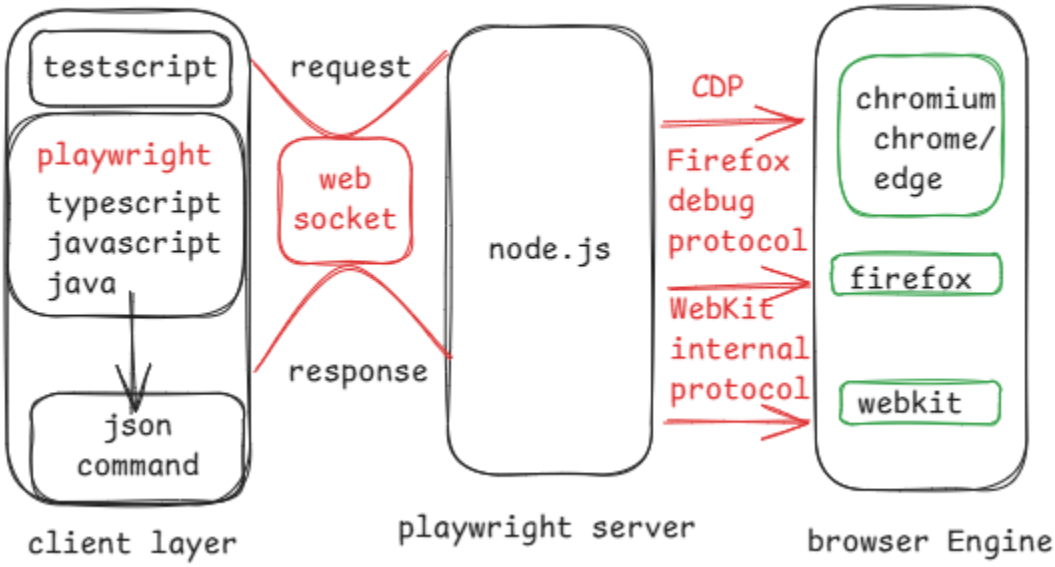
playwright is an open source node.js lib

Node.js=its an open source ,cross platform JavaScript runtime Enivroment

Feature

1)cross browser=(chromium)-chrome and edge ,firefox,webkit(safari)
2)cross platform=(macos,window,linux)
3)langauge=support JS/TS,Java,python, and c#
4)mobile web testing
5)Api testing
6)Auto-Wait(reduce flakiness)
7)Parallel exeution
8)Built in report
9)built in test runner
10)built in Assertion(validation)

Playwright Architecture



Test Script → Client Layer → WebSocket → Playwright Server → Browser Protocol → Browser

1. Test Script
This is where we write our automation code.
It gives the instructions to Playwright.
Languages supported: JavaScript, TypeScript, Java, Python, .NET.

2. Client Layer
It converts the Playwright command into a JSON command.
It acts as a translator.

3. JSON Command
It represents our Playwright commands in a structured way.

4. WebSocket
It acts like a communication channel between the client and the Playwright server.
It provides two-way communication (request and response).
The connection remains open until all the commands are executed.
Normally it closes once all the commands are executed.

5. Playwright Server
It acts as the brain.
It receives the commands from the client layer.
It runs on Node.js.
It handles things such as auto-waiting.

6. Browser Protocol
It is the communication language between the Playwright server and the browser engine.
It converts the Playwright request into the browser-specific language.

Browser	Protocol
Chromium (Chrome / Edge)	CDP (Chrome DevTools Protocol)
Firefox	Firefox debug protocol
WebKit (Safari)	WebKit internal protocol

7. Browser Engine
It is the technology which processes the web page.

How It Works: Example

Command written in the test script:

```
await page.click('#login');
```

The test script sends the command to the client layer.
The client layer converts it into a JSON command.
The first request builds the WebSocket connection between the client layer and the server layer.
The JSON command travels through the WebSocket to the Playwright server (Node.js).
The Playwright server converts the request into the browser-specific protocol (CDP, Firefox debug protocol or WebKit internal protocol).
The browser engine executes the click on the web page.
The response comes back along the same path to the test script.
Once all commands are executed, the WebSocket connection is destroyed.